

# Desarrollo de MLang

## Autores

*Lucas Campoli*

*Franco Agustín Ghigliani*

## Tabla de contenidos

|                                  |          |
|----------------------------------|----------|
| <b>Tabla de contenidos.....</b>  | <b>1</b> |
| <b>Introducción.....</b>         | <b>1</b> |
| <b>Modelo Computacional.....</b> | <b>2</b> |
| Dominio.....                     | 2        |
| Lenguaje.....                    | 2        |
| <b>Implementación.....</b>       | <b>4</b> |
| Frontend.....                    | 4        |
| Backend.....                     | 5        |
| Dificultades Técnicas.....       | 5        |
| <b>Futuras Extensiones.....</b>  | <b>6</b> |
| <b>Conclusiones.....</b>         | <b>6</b> |
| <b>Referencias.....</b>          | <b>6</b> |
| <b>Bibliografía.....</b>         | <b>7</b> |

# Introducción

En el siguiente documento se detallarán las distintas etapas de desarrollo del lenguaje de programación MLang, junto a posibles adiciones a futuro.

El objetivo de este proyecto es desarrollar un lenguaje declarativo que permita modelar y crear sistemas dinámicos de un videojuego para ser simulados en Machinations.io. Desde este sitio las simulaciones también pueden ser exportadas a Unity para asistir el desarrollo de videojuegos. El lenguaje permite establecer las propiedades de los diferentes nodos y sus conexiones, y definir los parámetros generales de la simulación; como su velocidad, la cantidad de steps a simular, entre otros.

La implementación satisfactoria de este lenguaje permite experimentar con diferentes sistemas comunes, tales como diseñar economías o procesos aleatorios que ocurren dentro de videojuegos, y sus variaciones de implementación, medirlos y balancearlos antes de implementar los mismos en un proyecto, minimizando tiempos de playtesting y facilitando la detección de sistemas desbalanceados.

El código fuente del compilador resultante de este trabajo puede ser encontrado en [https://github.com/Franco-A-Ghigliani/TLA\\_TP-MLang](https://github.com/Franco-A-Ghigliani/TLA_TP-MLang).

# Modelo Computacional

## Dominio

**Machinations** es una herramienta ampliamente utilizada para modelar, analizar y balancear sistemas dinámicos en videojuegos y simulaciones. Fue diseñada por Joris Dormans como un marco conceptual que combina la teoría de sistemas con las dinámicas de juego, permitiendo a los diseñadores crear diagramas interactivos que representan la lógica subyacente de un sistema o juego.

En esencia, **Machinations** es una plataforma visual que permite modelar y simular cómo los diferentes componentes de un sistema interactúan entre sí. Esto se realiza mediante **diagramas dinámicos**, que combinan conceptos como nodos, recursos y conexiones para mostrar cómo fluyen los datos (o recursos) a través del sistema. Es especialmente útil para:

1. **Simular mecánicas de juego** como progresión, economía, generación de recursos o interacción entre jugadores.
2. **Identificar desequilibrios** en sistemas complejos antes de implementarlos.
3. **Prototipar y experimentar** con diferentes configuraciones en tiempo real.

Para reducir la complejidad del proyecto, no todos los posibles componentes de Machinations fueron representados en el lenguaje, tales como traders o registros. Esto es ya que son componentes cuyos usos no son comunes ni frecuentes, y además no son necesarios para el desarrollo de una simulación suficientemente compleja. Tampoco se representaron las integraciones relacionadas al estilo, como pueden ser los colores o aspectos de los componentes en la interfaz gráfica de Machinations.

## Lenguaje

Se puede crear una **simulación** con sus respectivos argumentos:

- name
- stepInterval
- stepsToSimulate

Respecto a los tipos de nodos admitidos en el sistema y los argumentos que cada uno puede recibir, se describen a continuación las características y configuraciones específicas:

1. Todos los nodos tienen los **atributos básicos**:
  1. label: el nombre a mostrar en la simulación
  2. position: coordenadas (x, y) en las cuales mostrar el nodo al ser importado desde Machinations.
  3. activation: cuándo activar el nodo. Puede ser interactive (activa con click), passive (activa con señales de otro nodo), automatic (activa en cada step), o onstart (activa únicamente en el primer step). La única excepción es el nodo EndCondition, el cual no tiene este atributo
2. Los **Sources** son los generadores de recursos. Además tienen un atributo adicional resourceColor, lo que permite diferenciar los recursos para representar distintas cosas.

3. Los **Converters** transforman recursos -definidos por las conexiones entrantes- en otros. También tienen los atributos:
  1. resourceColor
  2. multipleConversion
4. Los **Gates** distribuyen los recursos entrantes entre los nodos conectados. La distribución de los recursos es determinada por las conexiones de recursos salientes. Además tiene los atributos:
  1. activationMode
  2. randomDistribution
5. Los **Drains** se deshacen de recursos. También tienen el atributo activationMode.
6. Las **Pools** son contenedores de recursos y sirven como medición de diversas cosas como, por ejemplo, cuántas monedas tiene el jugador en la simulación. Tienen además atributos:
  1. activationMode
  2. initialResources
  3. initialResourcesColor
  4. capacity
  5. numberDisplayThreshold
  6. drainOnOverflow
7. Los **Delays** sirven para retrasar el tránsito de recursos. Tienen también el atributo adicional isQueue.
8. Finalmente, las **EndConditions** al ser activados detienen la simulación. Representan un fin del juego (sea porque el jugador pierde, porque gana, o porque simplemente no tiene sentido continuar la simulación). No tienen ningún atributo adicional pero la condición que representan está definida por la fórmula de la conexión de estado entrante.
9. Mediante el uso de “=>” o “=#”, se pueden crear las **conexiones** entre los nodos para simular transferencias de recursos o condiciones respectivamente.
  1. Mediante conexiones de recursos, se simulan las transferencias de recursos entre los distintos nodos.
  2. Mediante conexiones de estado, se pueden mandar las señales para activar un nodo con activation=passive o alterar la fórmula de una conexión de recurso.

Además de lo mencionado anteriormente, se ofrecen métodos diseñados para simplificar y optimizar la experiencia del desarrollador:

1. Se pueden crear **Templates de nodos** a partir de un tipo de nodo preexistente. Es decir, definir los atributos de un nodo de ese tipo para luego poder ser reutilizado. También sirve para centralizar estos parámetros ya que al modificar los parámetros especificados en el Template se modifican todos los nodos creados a partir del mismo. Se pueden sobrescribir parámetros de un template en la declaración de un nodo de ese tipo.
2. Se pueden crear **Templates de Simulations** (circuitos enteros) especificando entradas y salidas. Estos Templates de Simulations pueden ser reutilizados como si fuera un nodo para tener su mismo comportamiento repetido en diversas partes de una misma simulación.
3. Se puede marcar el resto de una línea como un **comentario** con la keyword //
4. Se pueden definir **constantes** con la keyword const para luego ser utilizada en los parámetros de los distintos objetos.

# Implementación

## Frontend

Se han utilizado herramientas robustas como **Flex** y **Bison** para el análisis léxico y sintáctico, respectivamente.

### 1. Análisis Léxico con Flex

El analizador léxico fue diseñado para identificar los tokens básicos del lenguaje, tales como palabras clave (e.g., const, simulation), identificadores, números, y operadores ( $\Rightarrow$ ,  $\#$ ).

Flex se utiliza para dividir la entrada del usuario en **tokens** significativos. Estos tokens representan las unidades básicas del lenguaje, como tipos de nodos, valores numéricos, palabras clave y símbolos de control.

Se definieron patrones regulares en **Flex** que aseguran un reconocimiento eficiente y robusto de las unidades léxicas.

### 2. Análisis Sintáctico con Bison

Bison se encarga de interpretar la secuencia de tokens generada por Flex y validar si sigue las reglas de la **gramática** definida para el lenguaje. A partir de esta interpretación, construye un árbol de sintaxis abstracta (**AST**) que representa la estructura lógica de la simulación.

El analizador sintáctico define una gramática formal libre de contexto que modela las estructuras válidas del lenguaje. Esto incluye:

- **Definiciones** para declaraciones de nodos, conexiones, y simulaciones completas.
- **Reglas de precedencia** para operadores, garantizando una evaluación adecuada.
- **Construcción del AST (Árbol de Sintaxis Abstracta):** Cada producción de la gramática se diseñó para generar nodos en el AST, permitiendo representar la estructura lógica de los programas.

## Backend

El backend se centró en validar la semántica del programa y transformar el AST en un modelo ejecutable para generar las salidas requeridas por el dominio de Machinations.

### 1. Análisis Semántico

Una vez construido el AST, se realizaron validaciones semánticas para garantizar la coherencia del programa.

- **Validación de Tipos:** Verificación de compatibilidad en conexiones entre nodos y parámetros, asegurando que los valores sean adecuados para cada atributo.

- **Resolución de Referencias:** Identificación y sustitución de constantes y templates, con detección de referencias circulares.
- **Verificación de Reglas de Dominio:** Comprobación de restricciones específicas, como la presencia de atributos no válidos en ciertos nodos (e.g., activation en EndConditions).

## 2. Cómputo del AST

Una vez validado el AST, se realizaron operaciones de cómputo para crear las estructuras necesarias para la generación de código. Es importante aclarar que esta fase es importante ya que se encarga de computar todas las **expresiones matemáticas**, setear todos los **valores predeterminados** de aquellos atributos opcionales, y además **ordenar y compactar** la información.

- **Creación de estructuras:** Se generaron las estructuras necesarias y popularon acordeamente en base al AST validado.
- **Ordenamiento de información:** Se procesó la información para que el generador de código pueda generar el archivo CSV de manera correcta y eficiente.
- **Asignación de atributos predeterminados o transparentes:** Se asignaron todos aquellos atributos predeterminados en caso de que dichos atributos opcionales no hayan sido provistos. Además se asignó información relevante del backen como son las asignaciones de ID usados por Machinations.

## 3. Generación de Código

El compilador genera archivos compatibles con **Machinations.io**, traduciendo el modelo interno del programa a una representación CSV que puede ser importada directamente en la herramienta.

- **Exportación CSV:** Se generaron tablas en un archivo CSV que representan los nodos, conexiones, y configuraciones de la simulación.
- **Testing:** Para verificar la integridad del backend, se diseñó un conjunto de programas de prueba que comprueban casos de aceptación y rechazo, incluyendo simulaciones completas y nodos específicos.

## Dificultades Técnicas

Durante la implementación, surgieron desafíos que requirieron soluciones específicas:

- **Ambigüedad en la Gramática:** Algunas reglas de la gramática presentaban conflictos shift/reduce que se resolvieron utilizando precedencia explícita y reglas más detalladas.
- **Manejo de Recursos en C:** La gestión manual de memoria para estructuras dinámicas -como el AST, las tablas de símbolos, y las estructuras de la simulación computada- requirió especial atención para evitar fugas y accesos indebidos.

# Futuras Extensiones

El equipo ha pensado posibles adiciones al compilador entre las cuales se encuentran:

- Soporte para las estructuras de Machinations faltantes (Traders y Registers) para así poder modelar simulaciones aún más complejas.
- Soporte para modificar las propiedades gráficas de los elementos. Estos pueden ser los colores, el tipo de flecha que representa una conexión, el grosor de las líneas, etc.
- Un resaltador de sintaxis para mejorar la *Developer Experience*
- Ciclos, para así poder crear múltiples veces el mismo nodo sin necesidad de reescribir la misma declaración.
- Guardar posiciones relativas de los nodos de un Template:Simulation de manera que al tener múltiples instancias de la misma se mantenga la estructura gráfica.

# Conclusiones

El lenguaje de programación **MLang** proporciona una solución declarativa para modelar y simular sistemas dinámicos en videojuegos a través de la plataforma **Machinations.io**. Su diseño permite a desarrolladores y diseñadores de juegos experimentar y equilibrar sistemas complejos de manera eficiente antes de su implementación, optimizando así los tiempos de desarrollo y reduciendo la necesidad de extensas sesiones de playtesting.

Al centrarse en los componentes más relevantes y comunes de Machinations, como nodos, conexiones y parámetros generales de simulación, **MLang** logra captar la esencia de los sistemas dinámicos sin la carga de características menos utilizadas, como traders o registros. Esto facilita la curva de aprendizaje del lenguaje y permite enfocar los esfuerzos en las mecánicas clave de los sistemas.

**MLang** no sólo acelera el desarrollo de sistemas equilibrados, sino que también abre nuevas oportunidades para prototipar y experimentar mediante código. Al ofrecer esta flexibilidad programática, los desarrolladores pueden integrar simulaciones directamente en sus flujos de trabajo y probar configuraciones dinámicas sin depender exclusivamente de la interfaz gráfica de Machinations, lo que amplía el alcance de esta poderosa herramienta.

# Referencias

[https://github.com/Franco-A-Ghigliani/TLA\\_TP-MLang](https://github.com/Franco-A-Ghigliani/TLA_TP-MLang)

[machinations.io](https://machinations.io)

# Bibliografía

[machinations.io/docs](https://machinations.io/docs)

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd Edition). En *Addison-Wesley Longman Publishing Co., Inc. eBooks*.  
<https://dl.acm.org/citation.cfm?id=1177220>